# Task Manager API

Technical Design Report

Mário Prazeres · github.com/MPrazeres-1983

This document provides a complete technical walkthrough of the Task Manager API — a production-grade REST API built with Java 17 and Spring Boot 3. It covers every architectural decision, from the choice of framework to the security model, test strategy, and deployment approach. The goal is not just to describe what was built, but to explain why each choice was made.

## 1. Project Overview

The Task Manager API is a stateless RESTful web service that allows authenticated users to create, read, update, and delete personal tasks. Each user's data is fully isolated — a user can only access their own tasks, enforced at both the service and query level.

The project was built as a deliberate, structured introduction to Java fullstack development, applying the same engineering discipline used in previous Python projects: automated testing, CI/CD pipelines, containerisation, and clean architecture.

### 1.1 Core Features

- JWT-based stateless authentication (register and login)
- Full CRUD for tasks with per-user data isolation
- Filtering by status (TODO / IN_PROGRESS / DONE) and priority (LOW / MEDIUM / HIGH)
- Pagination on all list endpoints (default: 20 items, sorted by createdAt DESC)
- Partial update endpoint: PATCH /tasks/{id}/status for status-only changes
- Automatic timestamps (createdAt, updatedAt) via JPA Auditing
- Interactive API documentation via Swagger UI (SpringDoc OpenAPI)
- Global exception handler with RFC 7807 Problem Detail responses
- Bean validation on all request bodies with field-level error messages
- 20+ automated tests: unit (JUnit 5 + Mockito) and integration (MockMvc + H2)
- Multi-stage Docker build with non-root runtime user
- GitHub Actions CI: compile, test, Docker build

## 2. Architecture

The application follows a classic three-layer architecture: Controller, Service, and Repository. This separation is deliberate — it isolates HTTP concerns from business logic, and business logic from persistence. Each layer can be tested independently.

### 2.1 Layer Responsibilities

| Layer | Class(es) | Responsibility |
|---|---|---|
| Controller | AuthController, TaskController | HTTP handling, input validation, response mapping |
| Service | AuthService, TaskService | Business logic, orchestration, transaction boundaries |
| Repository | UserRepository, TaskRepository | Database queries via Spring Data JPA |
| Security | JwtAuthFilter, JwtService, SecurityConfig | Token validation, authentication context |
| Exception | GlobalExceptionHandler | Centralised error handling, RFC 7807 format |

### 2.2 Request Lifecycle

Every authenticated request follows this path:

- **1. JwtAuthFilter** intercepts the request, extracts the Bearer token, validates it, and sets the authentication context in Spring's SecurityContextHolder.
- **2. SecurityConfig** authorises the request based on the authenticated principal.
- **3. Controller** receives the request, validates the body with @Valid, extracts the authenticated user via @AuthenticationPrincipal, and delegates to the service.
- **4. Service** performs business logic, queries the repository, and returns a domain object.
- **5. Controller** maps the domain object to a DTO (TaskResponse record) and returns the appropriate HTTP status.
- **6. GlobalExceptionHandler** intercepts any exception and returns a structured Problem Detail response.

---

## 3. Data Model

The application has two JPA entities: User and Task, with a one-to-many relationship (one user has many tasks). The relationship is unidirectional from Task to User — tasks hold a foreign key to their owner, but users do not eagerly load their task list.

### 3.1 User Entity

```
@Entity
@Table(name = "users",
       uniqueConstraints = @UniqueConstraint(columnNames = "email"))
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    @Column(unique = true) private String email;
    private String password;       // BCrypt-encoded
    @CreatedDate private LocalDateTime createdAt;
    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)
    private List<Task> tasks;
}
```

### 3.2 Task Entity

```
@Entity
```

```
@Table(name = "tasks")
public class Task {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    @Column(columnDefinition = "TEXT") private String description;
    @Enumerated(EnumType.STRING) private Status status;   // TODO, IN_PROGRESS, DONE
    @Enumerated(EnumType.STRING) private Priority priority; // LOW, MEDIUM, HIGH
    private LocalDate dueDate;
    @CreatedDate    private LocalDateTime createdAt;
    @LastModifiedDate private LocalDateTime updatedAt;
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id") private User user;
}
```

**Design decisions:**

- Status and Priority are stored as strings (EnumType.STRING) not integers. This makes the DB human-readable and avoids ordinal bugs when enum order changes.
- LAZY fetching on the User relationship avoids N+1 query problems when listing tasks.
- @EnableJpaAuditing on the main class + @EntityListeners(AuditingEntityListener.class) handles createdAt/updatedAt automatically — no manual timestamp management.
- CascadeType.ALL with orphanRemoval=true on the User side means deleting a user automatically removes all their tasks — no orphan rows.

---

## 4. Security Model

Authentication is stateless, based on JSON Web Tokens (JWT). No server-side sessions are maintained — the token carries all authentication state. This is appropriate for REST APIs consumed by multiple clients.

### 4.1 Authentication Flow

- **Registration (POST /api/auth/register):** password encoded with BCrypt (strength 10), user saved, JWT generated and returned immediately.
- **Login (POST /api/auth/login):** AuthenticationManager verifies credentials via DaoAuthenticationProvider. On success, JWT generated and returned.
- **Subsequent requests:** JwtAuthFilter reads the Authorization header, validates the token signature and expiry, and sets the security context.

### 4.2 JWT Implementation

Tokens are signed with HMAC-SHA256 using a secret key of at least 256 bits (configurable via environment variable). Expiry is 24 hours by default (also configurable).

The JwtService uses the jjwt library (v0.12.x) with the modern fluent API. Token claims: subject (email), issuedAt, expiration.

### 4.3 User Isolation

Every task query includes the authenticated user's ID as a filter condition. A user cannot retrieve, update, or delete another user's tasks — even if they know the task ID. This is enforced at the service layer:

```
// TaskRepository
Optional<Task> findByIdAndUserId(Long id, Long userId);

// TaskService
private Task findTaskByIdAndUser(Long taskId, Long userId) {
    return taskRepository.findByIdAndUserId(taskId, userId)
        .orElseThrow(() -> new ResourceNotFoundException("Task not found"));
}
```

If a user attempts to access another user's task, they receive a 404 — not a 403. This avoids leaking information about resource existence.

---

## 5. API Design

### 5.1 Endpoint Reference

| Method | Endpoint | Auth | Status | Description |
|---|---|---|---|---|
| POST | /api/auth/register | No | 201 | Register — returns JWT |
| POST | /api/auth/login | No | 200 | Login — returns JWT |
| GET | /api/tasks | JWT | 200 | List tasks (paginated) |
| POST | /api/tasks | JWT | 201 | Create task |
| GET | /api/tasks/{id} | JWT | 200 | Get task by ID |
| PUT | /api/tasks/{id} | JWT | 200 | Full update |
| PATCH | /api/tasks/{id}/status | JWT | 200 | Status update only |
| DELETE | /api/tasks/{id} | JWT | 204 | Delete task |

### 5.2 DTOs as Java Records

All request and response objects use Java Records (introduced in Java 16). Records are immutable by default, concise, and carry built-in equals/hashCode/toString.

```
// Request DTO (record with Bean Validation)
public record TaskRequest(
    @NotBlank @Size(max = 200) String title,
    @Size(max = 2000)          String description,
    Status status,
    Priority priority,
    @FutureOrPresent LocalDate dueDate
) {}

// Response DTO with factory method
public record TaskResponse(Long id, String title, ...) {
    public static TaskResponse from(Task task) { ... }
}
```

### 5.3 Error Handling — RFC 7807 Problem Detail

All error responses follow the RFC 7807 Problem Detail standard, supported natively by Spring Boot 3. This gives clients consistent, machine-readable error payloads:

```
{
  "type": "about:blank",
  "title": "Validation Error",
  "status": 400,
  "detail": "Validation failed",
  "timestamp": "2025-03-01T10:00:00Z",
  "errors": {
    "title": "Title is required",
    "dueDate": "Due date must be today or in the future"
  }
}
```

## 6. Testing Strategy

The test suite is structured in two layers: unit tests (fast, isolated, no Spring context) and integration tests (full Spring context, H2 in-memory database, MockMvc).

| Test Class | Type | Coverage |
|---|---|---|
| JwtServiceTest | Unit | Token generation, extraction, validation, expiry |
| AuthServiceTest | Unit | Register (ok + duplicate), login (ok + bad credentials) |
| TaskServiceTest | Unit | CRUD, defaults, 404, status update, user filtering |
| AuthControllerTest | Integration | HTTP register/login — 201, 409, 400, 401 |
| TaskControllerTest | Integration | HTTP CRUD — auth, isolation, 400, 404, 204 |

### 6.1 Unit Test Design (Mockito)

Service tests use Mockito to mock all dependencies. This keeps tests fast (milliseconds), deterministic, and focused on the unit under test. Example:

```
@ExtendWith(MockitoExtension.class)
class AuthServiceTest {
    @Mock private UserRepository userRepository;
    @Mock private PasswordEncoder passwordEncoder;
    @InjectMocks private AuthService authService;

    @Test void register_shouldThrowIfEmailExists() {
        when(userRepository.existsByEmail("exists@test.com")).thenReturn(true);
        assertThatThrownBy(() -> authService.register(request))
            .isInstanceOf(ConflictException.class);
        verify(userRepository, never()).save(any());
    }
}
```

### 6.2 Integration Test Design (MockMvc + H2)

Controller tests load the full Spring context with @SpringBootTest and use @ActiveProfiles('test') to switch to H2 in-memory database. MockMvc sends real HTTP requests through the full filter chain, including JWT authentication:

```
@SpringBootTest @AutoConfigureMockMvc @ActiveProfiles("test")
class TaskControllerTest {
    @Test void getTasks_shouldIsolateByUser() throws Exception {
        // User 1 creates a task
        mockMvc.perform(post("/api/tasks").header("Authorization", "Bearer " + token) ...);
        // User 2 sees empty list
        String otherToken = registerAndGetToken("other@test.com", "pass");
        mockMvc.perform(get("/api/tasks").header("Authorization", "Bearer " + otherToken))
            .andExpect(jsonPath("$.totalElements").value(0));
    }
}
```

## 7. Containerisation

### 7.1 Multi-Stage Dockerfile

The Dockerfile uses a two-stage build to minimise the final image size. The builder stage uses the full Maven image to compile and package the application. The runtime stage uses a minimal JRE Alpine image (~180MB vs ~600MB for the full JDK).

```
# Stage 1: Build
FROM maven:3.9.6-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -q   # Cache layer
COPY src ./src
RUN mvn clean package -DskipTests -q

# Stage 2: Runtime (minimal)
FROM eclipse-temurin:17-jre-alpine
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser                          # Non-root for security
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

### 7.2 Docker Compose

Docker Compose orchestrates the API and PostgreSQL database together. A health check on the DB service ensures the API only starts once Postgres is ready, avoiding connection errors on cold start. All sensitive configuration (passwords, JWT secret) is injected via environment variables — never hardcoded.

```
services:
  db:
    image: postgres:15-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
  api:
    build: .
    environment:
      DB_URL: jdbc:postgresql://db:5432/taskmanager
      JWT_SECRET: ${JWT_SECRET}    # From environment, never in source
    depends_on:
      db:
        condition: service_healthy
```

## 8. CI/CD Pipeline

The GitHub Actions pipeline runs on every push to main/develop and every pull request. It has two sequential jobs:

- **build-and-test:** Checkout → JDK 17 setup → Maven cache → Compile → Run all tests (H2, no external DB needed) → Upload test report artifact

- **docker-build:** Builds the Docker image and verifies it exists — confirms the Dockerfile is always valid.

Maven dependency caching is configured to avoid re-downloading dependencies on every run, keeping pipeline time under 2 minutes for a clean build.

---

## 9. Deployment Guide

### 9.1 Docker Compose (Local / Staging)

```
# Clone and run
git clone https://github.com/MPrazeres-1983/task-manager-api.git
cd task-manager-api
docker-compose up --build

# API:     http://localhost:8080
# Swagger: http://localhost:8080/swagger-ui.html
```

### 9.2 Environment Variables

| Variable | Default | Description |
|---|---|---|
| DB_URL | jdbc:postgresql://localhost:5432/taskmanage | PostgreSQL JDBC URL |
| DB_USER | postgres | Database username |
| DB_PASS | postgres | Database password |
| JWT_SECRET | (see application.yml) | Min 256-bit signing key — change in production |
| JWT_EXPIRATION_MS | 86400000 | Token validity in ms (default: 24h) |
| PORT | 8080 | Server port |

### 9.3 Production Checklist

- Set JWT_SECRET to a randomly generated 256-bit key (not the default)
- Set spring.jpa.hibernate.ddl-auto to validate (not update) in production
- Enable HTTPS — the API should never serve tokens over plain HTTP
- Configure database connection pooling (HikariCP defaults are reasonable)
- Add rate limiting on /api/auth/** endpoints to prevent brute-force attacks

---

## 10. Key Learning Points

This project was built as a first structured Java application, transitioning from Python. These are the most significant concepts internalised through building it:

- **Spring's inversion of control** — dependency injection via constructor (not @Autowired on fields) makes dependencies explicit and simplifies testing. @RequiredArgsConstructor (Lombok) generates the constructor automatically.
- **The Repository pattern** — Spring Data JPA generates query implementations from method names (findByUserIdAndStatus). For complex queries, @Query with JPQL is cleaner than Criteria API for simple

cases.

- **Records as DTOs** — Java Records eliminate boilerplate (no getters, no constructors, immutable by design). The factory method pattern (TaskResponse.from(task)) keeps mapping logic in the DTO, not the service.
- **Stateless auth with JWT** — the OncePerRequestFilter pattern for the auth filter guarantees exactly one execution per request, avoiding double-processing.
- **Test profiles** — @ActiveProfiles('test') with a separate application-test.yml using H2 means tests run anywhere without a real database. Profiles are the right abstraction for environment-specific config.
- **Security defaults are restrictive** — Spring Security 6 denies everything by default. The SecurityFilterChain bean explicitly permits only public endpoints; everything else requires authentication.

---

Mário Prazeres · github.com/MPrazeres-1983 · marioprazeres@msn.com